

NYPC Rookie Qualification Round Replay

닉네임: tkgkb

사용한 LLM: OpenAI ChatGPT Plus, GPT-5.6 Thinking

이번 대회에서는 ChatGPT를 사용했다. 주로 내가 생각한 방법을 코드로 옮기다가 막혔을 때 자료구조나 계산식을 정리하고, 서로 다른 제출 파일의 차이를 찾는 데 이용했다. 답변을 그대로 믿고 제출한 것은 아니고, 컴파일과 직접 만든 시뮬레이터로 다시 확인했다. 실제로 AI가 정리한 코드를 급하게 반영했다가 세미콜론이 빠지거나 코드가 망가진 적도 있었고, 문제 규칙과 맞지 않는 계산을 AI가 진행한 적도 있었다. 그런 일이 몇 번 생긴 뒤부터는 한번에 코드를 크게 바꾸기보다 변경한 부분을 직접 비교하고 작은 입력 그리고 흐름을 따라가며 확인했다.

처음 문제를 읽었을 때는 꽃에서 나오는 씨앗의 개수와 굴이 원하는 개수만 얼추 비슷하게 맞추면 된다고 생각했다. 그런데 실제로는 개수만 맞추는 것으로 끝나지 않았다. 같은 칸에 씨앗이 너무 많이 남으면 과부하가 되고 그 칸으로 방금 들어온 씨앗이 다시 되돌아간다. 굴에도 매 초에 하나만 저장할 수 있어서 여러 경로가 같은 굴에 비슷한 시간에 도착하면 마지막 씨앗이 저장되는 시간이 늦어져버렸다. 행을 많이 사용하면 경로를 만들기는 편하지만 비용의 첫 항인 2^{R-C} 가 빠르게 커진다. 반대로 행을 너무 아끼면 경로가 겹치거나 씨앗이 막힐 가능성이 높았다. 결국 오차 E , 손실 L , 지연 D , 행 수 R 을 따로 줄이는 것이 아니라 서로 영향을 주는 값으로 봐야 했다.

가장 처음 제출한 #113186에서는 규칙을 거의 그대로 시뮬레이션하고, 격자의 칸을 무작위로 바꾸는 방식으로 좋은 보드를 찾으려고 했다. 각 칸에 다람쥐나 햄스터를 놓고, 매 초 씨앗이 생성되고 이동하고 되돌아가는 과정을 전부 계산했다. 제대로만 탐색할 수 있다면 가장 직접적인 방법이라고 생각했지만, 보드 하나를 평가하는 데도 시간이 많이 들었다. 더 큰 문제는 무작위로 바꾼 보드 대부분이 전혀 쓸 수 없는 상태였다는 점이다. 어느 한 칸만 잘못 바뀌어도 씨앗이 막히거나 엉뚱한 곳으로 가서, 제한 시간 안에 괜찮은 후보를 거의 찾지 못했다. 이 제출 이후에는 격자 자체를 처음부터 탐색하는 방법을 포기하고, 먼저 씨앗을 어느 굴로 보낼지 계산한 다음 그 결과를 격자에 옮기는 방식으로 생각을 바꿨다.

#113481에서는 점수가 좋지 않더라도 항상 정상적으로 작동하는 기준 코드를 만들었다. A_i 와 B_i 를 각각 정렬해 크기가 비슷한 꽃과 굴을 하나씩 연결하고, 꽃마다 서로 다른 행을 사용했다. 먼저 아래로 내려간 뒤 목표 열까지 옆으로 이동하고, 다시 굴까지 내려가는 단순한 경로였다. 한 꽃의 씨앗을 여러 굴로 나누지 못해 오차는 컸지만, 경로가 서로 겹치지 않아 출력 오류가 거의 없었고 직접 확인하기도 쉬웠다. 비용은 1,411,182였다. 이후에 새로운 방법을 만들 때마다 이 코드도 같이 후보로 남겨 두었다. 새 코드가 과부하나 손실을 만들면 점수가 훨씬 나빠질 수 있었기 때문에, 안전한 기준이자 템플릿으로 써먹을 수 있는 코드가 필요했다.

그다음에는 한 꽃의 씨앗을 여러 굴로 나누려는 시도를 계속했다. #113531에서는 행 전체를 왼쪽이나 오른쪽으로 흐르게 만들고, 중간에 다람쥐가 씨앗을 아래와 옆 방향으로 번갈아 보내도록 했다. 코드 안에서 계산한 총량은 괜찮아 보였지만 실제 규칙대로 돌리면 다람쥐가 다음에 사용할 방향과 씨앗이 도착하는 시간이 서로 꼬여버렸다. 같은 양을 보내더라도 몇 초에 걸쳐 들어오는지가 다르면 결과가 완전히 달라졌다. 바로 다음 제출에서는 다시 기준 코드로 돌아갔고, #114294에서는 입력의 씨앗 수를 작게 줄여 탐색하는 방법도 테스트해보았다. 하지만 두 방향 분할은 단순히 정확한 절반이 아니라 $\lceil A_i/2 \rceil$ 과 $\lfloor A_i/2 \rfloor$ 로 나뉘기 때문에, 값을 줄이는 과정에서 훌쩍이 바뀌면 작은 문제와 원래 문제가 다르게 움직였다. 이때부터 씨앗의 총량만 보는 대신 각 꽃에서 생기는 주기적인 조각을 그대로 계산해야 한다고 생각했다.

풀이 방향이 처음으로 크게 바뀐 제출은 #116336이었다. 한 꽃의 씨앗을 두 방향으로 보내면 두 개의 조각이 생긴다고 보고, 전체 $2C$ 개의 조각을 어느 굴에 배정할지 먼저 정했다. 조각의 목적지를 서로 바꾸면서 굴별로 오차를 줄이고, 그 결과를 실제 경로로 만들었다. 같은 굴로 가는 씨앗의 도착 시간이 겹치면 경로를 한 칸 더 돌아가게 해서 훌쩍을 바꾸기도 했다. 이전보다 목표량을 훨씬 세밀하게 맞출 수 있었지만, 배정이 좋아도 경로를 만들 수 없는 경우가 많았다. 어떤 조각은 숫자상으로는 좋은 위치에 들어가지만 다른 경로를 가로막아 행을 많이 사용하게 했다. 이때 배정 문제와 경로 배치 문제가 따로 있는 것이 아니라는 점을 알게 됐다.

#116339부터 #116559까지는 이 구조를 빠르게 만들고 오류를 고치는 과정이었다. 실제 경로를 매번 만들어 보는 대신 경로가 지나가는 구간의 밀도나 도착 시간의 훌쩍을 이용해 빠른 점수를 만들었고, 조각 두 개를 바꿀 때 모든 굴을 다시 계산하지 않고 영향을 받은 굴만 갱신하도록 바꿨다. 그 과정에서 입력문 뒤의 세미콜론이 빠진 제출, 현재 점수와 최고 점수를 같은 변수로 사용한 제출, 입력 부분에 잘못된 `return`이 들어간 제출도 있었다. 당시에는 높은 점수를 받고자(적은 비용으로 만들고자)수정할 때마다 제출을 너무 급하게 했고, 작은 문법 수정과 알고리즘 변경이 섞여 있었다. 이 구간의 제출들은 각각 길게 설명할 만한 새로운 아이디어라기보다, 코드를 안정시키는 과정에 가까웠다. 마지막에는 실제로 쓰는 칸만 점유 처리하고 탐색 시간을 1.95초로 줄여 (W)TLE를 면했지만, 여전히 완성된 보드를 규칙대로 검사하지 않는다는 문제가 남아 있었다.

#116694에서는 코드를 크게 수정하고 다시 갈아엎었다. 두 방향만 사용하던 구조를 $K = 2, 3$ 으로 일반화하고,

한 꽃을 K 방향으로 나눌 때 각 조각의 크기를

$$\frac{A_i + K - 1 - k}{K}$$

의 나눗셈으로 계산했다. 큰 조각부터 부족한 굴에 넣고, 굴의 사이에서 조각을 교환하며 오차를 줄였다. 무엇보다 이 제출부터 완성된 보드를 실제 규칙에 가깝게 돌리는 `simulate_board`를 코드 안에 넣었다. 이전에는 예상 점수가 좋은 후보를 그대로 선택했지만, 이제는 씨앗이 실제로 모두 도착하는지와 마지막 저장 시간이 언제인지 확인한 뒤 선택할 수 있었다. 처음으로 계산상 좋은 배정과 실제로 좋은 보드를 같은 코드 안에서 비교할 수 있게 된 점이 가장 큰 변화였다.

다만 전체 보드를 계속 복사하고 모든 칸을 훑어대느라 후보를 많이 만들 수 없었다. 그래서 #116716에서는 행의 점유 상태를 비트마스크로 저장하고, 보드 전체가 아니라 바뀐 칸만 `RouteEdit` 목록에 기록했다. 꽃 하나를 놓을 때도 첫 번째 가능한 위치를 바로 쓰지 않고 여러 `PlacementPlan`을 비교했다. 사용 중인 칸만 시뮬레이션하고 모든 씨앗이 저장되면 일찍 종료하도록 바꾸면서 속도는 많이 빨라졌다. 하지만 총 비용은 1,411,177로 기존 코드와 거의 차이가 없었다. 코드는 훨씬 복잡해졌는데 점수는 그대로여서 가장 허탈하고 어이없던 제출 중 하나였다. 배정 자체는 좋아졌지만 그 배정을 실제 보드로 만드는 성공률이 낮았고, 특히 가장자리 꽃에서 네 방향 분기를 만드는 것이 어려웠다. 이 결과를 보고 실행 속도만 높이는 것으로는 부족하고, 배정 단계부터 실제로 놓을 수 있는 모양을 고려해야 한다고 판단했다.

이후 #116758, #116790, #116962, #117033에서는 $K = 4$ 배정을 실제 보드에 하는데 집중했다. 단순한 무작위 교환 대신 가장 좋아지는 교환을 찾거나 세 조각의 목적지를 한 번에 돌렸고, 분기점을 꽃 바로 아래가 아니라 안쪽 열로 옮긴 뒤 네 방향으로 나누는 방법도 사용했다. $C = 5$, $K = 4$ 인 경우에는 20개 조각을 다섯 굴에 네 개씩 넣는 배정을 비트마스크로 더 정확하게 찾았다. 이때부터 목적지의 숫자뿐 아니라 경로의 모양까지 배정 점수에 조금씩 포함하기 시작했다.

중간에는 이미 찾은 보드를 코드에 저장해 같은 입력이 나오면 바로 출력하는 방법도 사용했다. #118640에서는 35개, #118875에서는 40개의 보드를 저장했다. 같은 입력에서는 빠르고 안정적이었지만 소스 대부분이 긴 문자열이 되었고, 입력이 조금만 달라지면 아무 도움이 되지 않았다. 몇 개만 남겨 보기도 하고 보드 생성기를 다시 작성하기도 했지만, 이 방식으로 얻은 점수는 일반적인 풀이가 좋아진 결과라고 보기 어려웠다. 당시에는 중간 평가의 결과를 올리는 데 집중해서 이런 방법도 사용했지만, 나중에는 특정 입력용 출력을 줄이고 일반 생성기를 다시 개선하는 쪽으로 돌아갔다.

#120964부터는 보드 생성 과정을 뼈대와 굴로 이어지는 끝부분으로 나눴다. 꽃에서 씨앗을 나누는 구조를 먼저 놓고, 남은 말단을 굴까지 연결했다. 이미 만들어진 세로 경로를 재사용하면 행을 줄일 수 있었지만, 앞에서 연결한 경로가 뒤의 경로를 막는 문제가 생겼다. #122985에서는 코드를 짧게 압축해 보았으나 한 줄에 여러 동작이 들어가 직접 검증하기가 너무 어려워졌다. 그 뒤 다시 함수 이름과 역할을 분명하게 나눴다. 이 구간에서는 새로운 수학적 관찰보다, 실패했을 때 배치를 되돌리고 여러 순서로 경로를 연결하는 구현을 계속 다듬었다.

비용이 크게 감소하기 시작한 것은 굴 앞의 대기 시간을 따로 계산하면서부터였다. #125713에서는 각 조각의 첫 도착 시각과 마지막 도착 시각을 이용해, 한 굴로 들어오는 속도가 초당 하나보다 빠른 구간에 대기열이 얼마나 생기는지 예상했다. 이전에는 굴이 최종적으로 받는 양만 맞으면 된다고 생각했지만, 같은 양이라도 도착 구간이 겹치면 D 가 크게 늘어났다. 그래서 오차와 예상 지연 중 큰 값을 배정의 점수로 사용했다. 예상값이 완전히 정확하지는 않았기 때문에 최종 후보는 다시 전체 시뮬레이션으로 확인했다. 같은 코드를 다시 낸 #125730은 별도의 변화가 없었고, #125760에서는 입력에 따라 $K = 2, 3$ 도 함께 시험했다. 목표량의 모양이 단순한 경우에는 무조건 네 방향으로 나누는 것보다 짧은 보드를 만들 수 있었다.

#125875에서는 굴에 도착하는 시간을 비트마스크로 저장하는 자유 라우터를 만들었다. 같은 굴에 같은 위상으로 두 스트림이 들어오지 않도록 이미 만든 세로 경로를 재사용하거나 일부러 한 칸을 더 돌아가게 했다. 홀짝만 보던 이전 방식보다 충돌을 자세히 확인할 수 있었지만, 서로 다른 주기가 섞이면 짧은 비트마스크만으로 모든 시간을 정확히 나타내기 어려웠다. 그래도 이 라우터를 만들면서 굴에 도착하는 시각을 경로의 부가 정보가 아니라 핵심 상태로 다루게 됐다.

가장 큰 개선이 나온 제출은 #126974였다. 주기 4로 들어오는 스트림의 도착 시간을 이용해 굴의 대기량을 계산하고, 마지막 씨앗이 들어온 뒤 남은 씨앗을 모두 저장하는 시간까지 구했다. 굴마다 반드시 네 조각을 받게 하지 않고, 필요한 굴에 조각 수를 다르게 배정하는 방법도 시험했다. 완성된 보드에서는 합류 지점과 작은 연결 구조, 세로로 내려가는 위치를 바꾸어 행 수와 지연을 줄였다. 이 제출의 총 비용은 89,834였고, 처음의 약 140만에서 처음으로 눈에 띄게 내려갔다. 여러 달라진 부분 중에서도 오차만 줄이지 않고 굴 앞 대기 시간과 실제 보드 모양을 같이 본 것이 가장 큰 이유라고 생각한다. 총 비용이 갑자기 내려간 것을 확인했을 때는 그동안 만든 복잡한 공식보다 굴이 한 초에 하나만 저장한다는 단순한 규칙을 늦게 반영했다는 생각이 들었다.

마지막 #129924에서는 끝까지 안전한 일대일 보드를 남겨 두고, 실제 시뮬레이션을 통과한 후보만 출력했다.

전체 과정을 돌아보면 가장 많은 시간을 쓴 부분은 조각의 오차를 몇 개 더 줄이는 계산이 아니라, 그 조각을 실제

격자 안에서 막히지 않게 보내는 일이었다. 특히 #116716에서 코드를 크게 최적화하고도 비용이 거의 그대로였을 때는 어디를 더 고쳐야 할지 잘 보이지 않았다. 반대로 #126974에서 굴 앞 대기 시간을 제대로 계산하자 비용이 크게 줄었다. 문제 설명에 처음부터 적혀 있던 굴의 처리 속도를 알고 있었지만, 오차를 줄이는 데 집중한 나머지 실제 목적함수에 미치는 영향을 낮게 생각한 것이다.